CS4475 Fall 2026
Alexa Ramirez-Salinas
aramirezsa@miners.utep.edu

Lab Report
Submitted on Sept.23, 2026

PA 1: Implementing xv6 User Commands and System Calls

Task 1. (5 pts) Implement the uptime utility
My code is in **user/uptime.c** on the pa1 branch, and I added $U/_uptime\ to UPROGS in the Makefile.

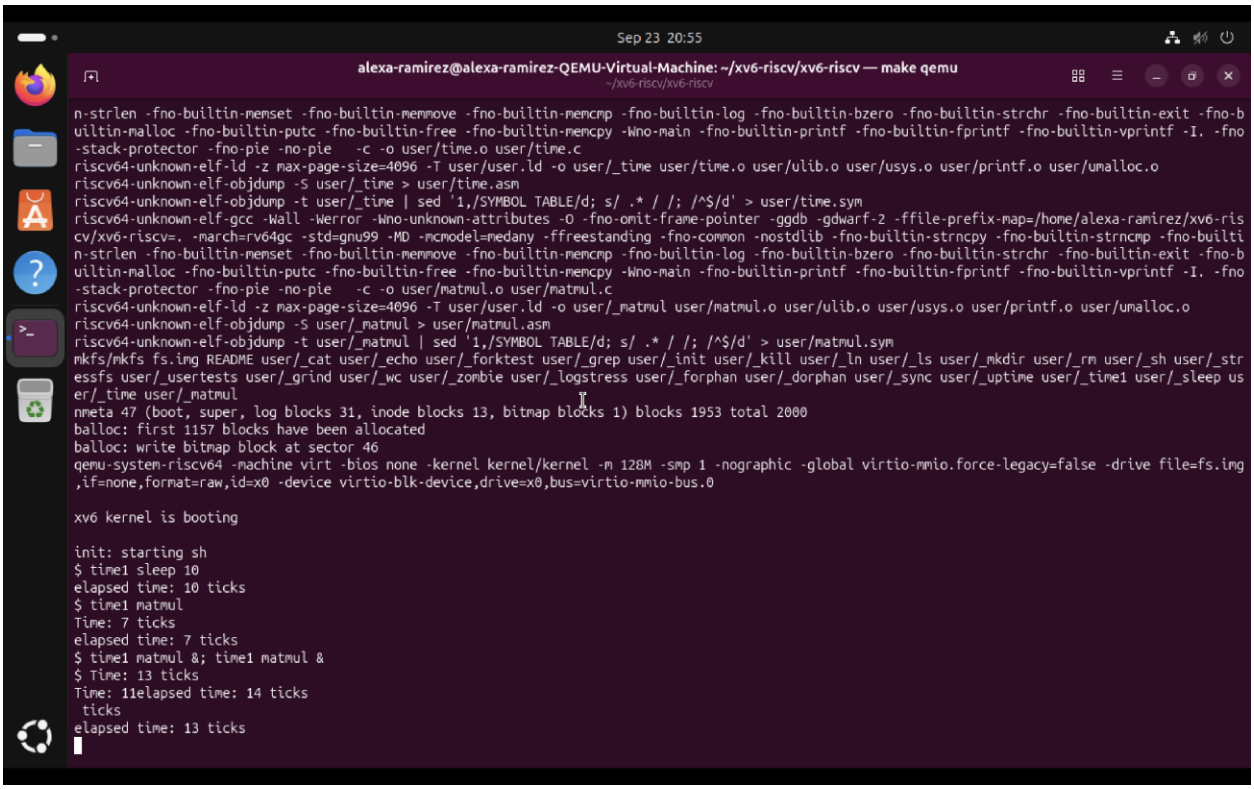
Screenshot of Result of Running uptime:



Task 2. (10 pts) Implement a time1 command that reports elapsed time.
My code is in **user/time1.c** on the pa1-syscalls branch, and I added $U/_time\ to UPROGS. time1 forks a child, execs the given command in the child, calls uptime() before the fork and again after wait() returns, and prints the difference.

The Time: line above comes from matmul.c itself, which prints its own timing. So, time1 only prints the elapsed time: line.

Screenshot of Result of Running time1:



Task 3. (10 pts) Keep track of how much cputime a process has used.
For this task I changed three files:

- **kernel/proc.h** - here I added an int cputime; field to structure proc, next to killed/xstate/pid, to hold the number of ticks the process has used.
- **kernel/proc.c** - here I initialized p->cputime = 0 in allocproc() when a new proc slot is claimed, and also reset it to 0 in freeproc() so a reused slot doesn't inherit a stale value from a previous process.
- **kernel/trap.c** - in both usertrap() and kerneltrap(), wherever*dev == 2* (this means a timer interrupt fired and the process is about to be preempted via yield()), here I incremented cputime by 1 right before calling yield(). I did it in both places because a process can get timer-preempted either while running in user mode (usertrap()) or while running in the kernel mid-syscall (kerneltrap()), and I wanted both cases counted.

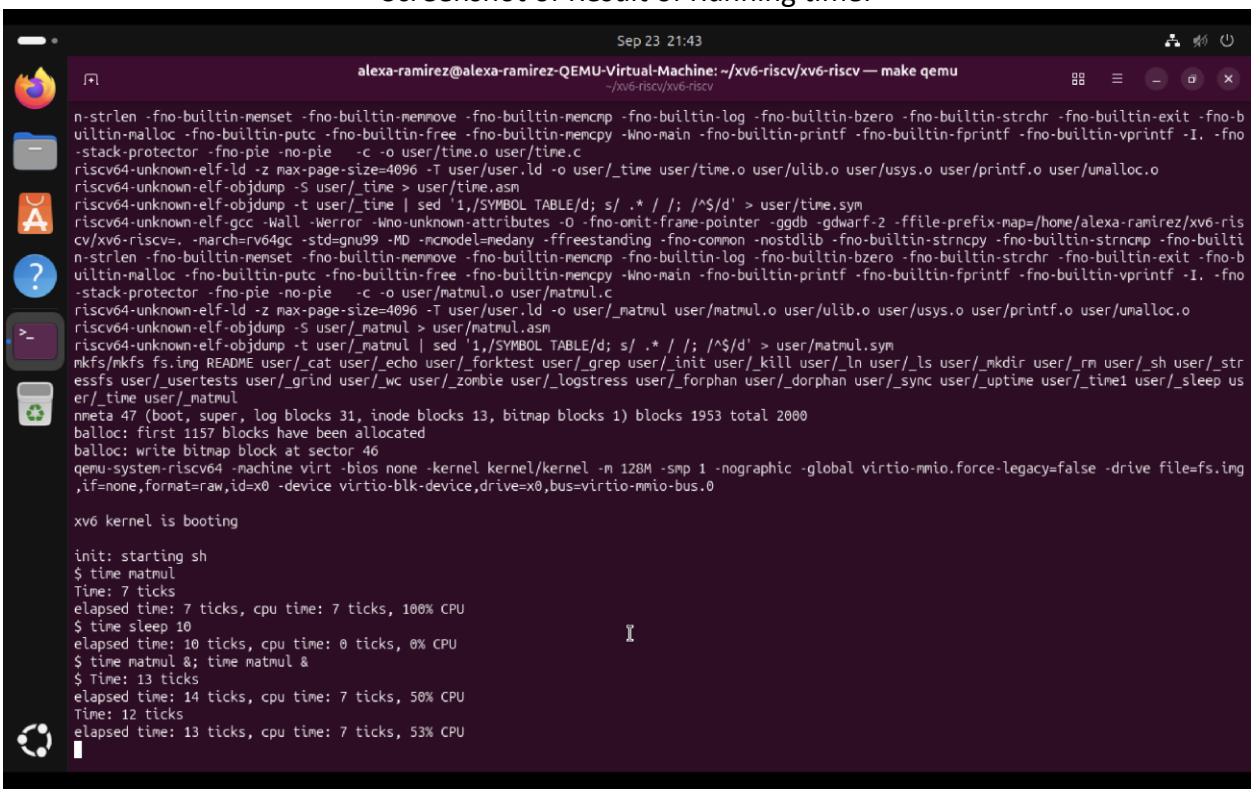
Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.
For this task I changed the following files:

- **kernel/pstat.h** (new file) - this defines struct rusage { uint cputime; };
- **user/user.h** - added struct rusage; forward declaration and the protorype int wait2(int**,** struct rusage*);
- **user/usys.p1** - added entry("wait2"); to generate the syscall stub.
- **kernel/syscall.h** - added #define SYS_wait2 23.

- **kernel/syscall.c** - added external uint64 sys_wait2(void); and the corresponding entry in the syscalls[] table.
- **kernel/sysproc.c** - added sys_wait2(), which pulls the two pointer arguments with argaddr() and calls kwait2().
- **kernel/proc.c** - added #include “pstat.h”, and added kwait2(), copied from kwait() with an extra block that copies out a struct rusage (currently just cputime) to the second pointer argument before freeing the zombie child.
- **kernel/defs.h** - added the prototype int kwait2(uint64, uint64); next to kwait.

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.
My code is in **user/time.c** on the pa1-syscalls branch, added to UPROGS as $U/_time\. It works like time1 but calls wait2() instead of wait(), then computes elapsed = end – start, cpu = ru.cputime, and percent = (cpu * 100) / elapsed.

Screenshot of Result of Running time:



Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

This assignment really helped me understand how xv6 actually measures time, since before this I never thought about what "CPU time" even means at the kernel level. Walking through uptime, time1, and time step by step showed me how a lot of what feels obvious in a normal Linux terminal (like /bin/time) actually depends on the kernel counting timer interrupts one tick at a time. Adding wait2() also taught me how many files a single new system call touches, I didn't expect to have to edit seven or eight files (syscall.h, syscall.c, usys.pl, sysproc.c, proc.c, defs.h, user.h) just to add one function that basically copies wait().

The hardest part was kwait2(). I made a few small mistakes, I miscounted the arguments to copyout(), forgot to reset pp->parent before calling freeproc(), and tried to use the standard xv6 sleep(chan, lock) signature when this version splits it into sleep_prepare() and sleep(). None of these caused a crash right away, which made them harder to notice, but once I compared my code line by line against the working kwait() function I already had, I found and fixed all three pretty quickly.

My use of AI -
I also used an AI assistant (Claude) as a study aid throughout this assignment, mainly to help me understand what each task was actually asking for before I started writing code. For example, when I was stuck on kwait2(), I described what I had written and the AI walked through the existing kwait() function with me line by line to explain what each part was doing (like why p->sz is needed in copyout(), and why pp->parent needs to be reset before freeproc()), which helped me catch my own mistakes instead of just being given a fix. Similarly for time.c, the AI helped me understand why the percentage calculation and rusage struct needed to work the way they do rather than just handing me a formula. For instance, it explained why cputime can be less than elapsed time when two processes share one CPU, which made it easier for me to explain the results in this report.

Extra Credit (5 points). Discuss limitations of our time command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).
One limitation is tick resolution, since uptime and cputime only count in whole ticks, making short-running programs look less accurate just from simple rounding. On top of that, our time program only tracks the direct child process it creates and waits on, so if that program starts its own sub-processes, any CPU time they use goes completely untracked by wait2(). Real Linux time handles this by adding up CPU usage across all child processes, but our simple version does not do that.